



MULTI-LAYER PERCEPTRON

INT301 Bio-computation, Week 4, 2025



多层感知机是一个由多个感知机构成的分层结构，使其能够学习非线性的函数映射，用来表示非线性决策边界。

Multilayer Perceptrons

- The **multilayer perceptron** (**MLP**) is a hierarchical structure of several perceptrons, which overcomes the shortcomings of the single-layer networks.
- The MLP neural network is able to learn nonlinear function mappings. MLP能表示非线性函数的原因是构成其神经元时，必须使用非线性的激活函数
 - learning a rich variety of nonlinear decision surfaces.
- Nonlinear functions can be represented by MLPs with units that use nonlinear activation functions.
 - Multiple layers of cascaded linear units still produce only linear mappings! 即使你将多个使用线性激活函数的单元堆叠在一起，最终整个网络仍然只能产生线性映射。

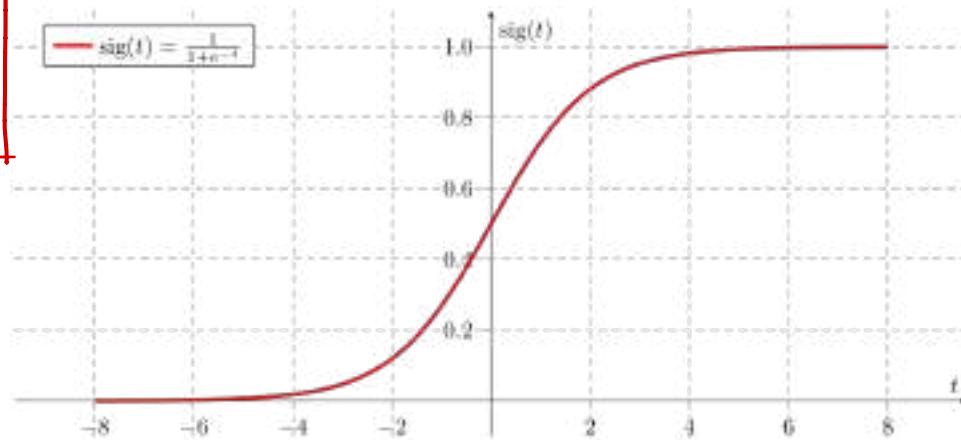
Differentiable Activation Functions

训练多层感知机的一个核心要求, 它的激活函数必须是可导的, 连续的, 非线性的

- Training algorithms for MLP require *differentiable, continuous nonlinear activation functions*.
- Such a function is the *sigmoid function*:

输出

$$o = \sigma(s) = \frac{1}{1 + e^{-s}}$$



where s is the sum:

输入的加权和

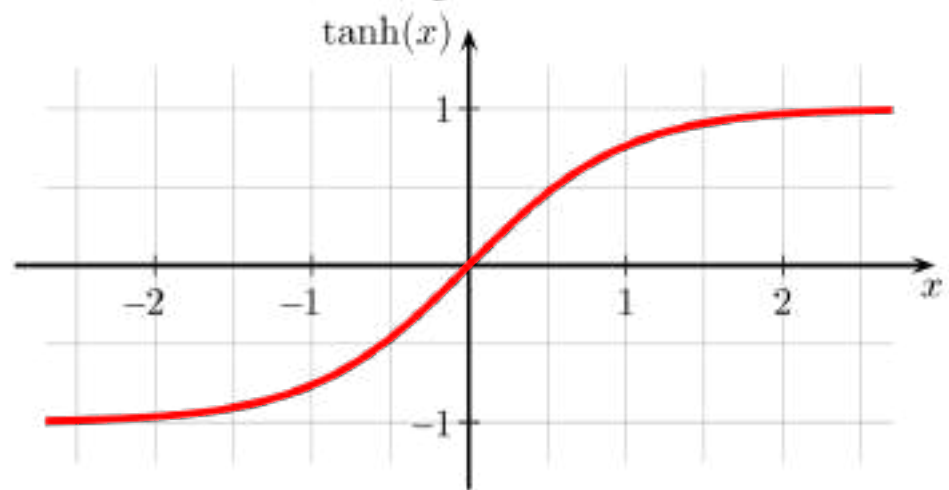
$$s = \sum_{i=0}^d w_i x_i$$

which is the products from the weights w_i and the inputs x_i . 该函数将任意实数输入映射到 (0,1) 的输出区间.

Differentiable Activation Functions

- Another nonlinear function often used in practice is the hyperbolic tangent. 双曲正切函数

$$o = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

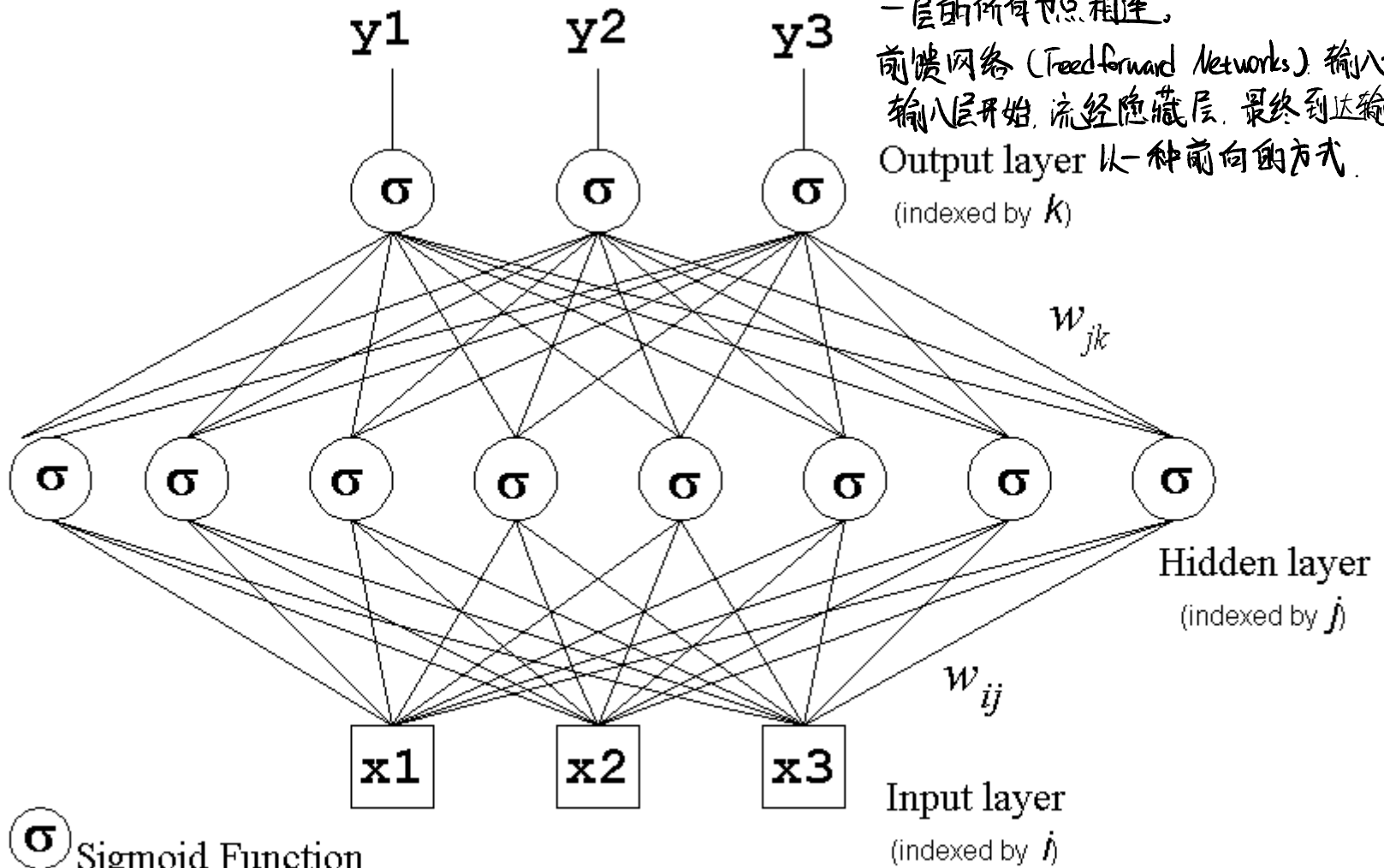


- The mean of tanh is 0

将输入映射到 $(-1, 1)$ 的区间内

tanh 的均值是 0, 因为其输出是关于原点对称

Multilayer Network Structure



高度连接的拓扑结构: MLP 中的一个节点会与下一层的所有节点相连。

前馈网络 (Feedforward Networks): 输入信号从输入层开始, 流经隐藏层, 最终到达输出层。

Output layer 以一种前向的方式。
(indexed by k)

内层公式: $G(\sum_{i=1}^I w_{ij}x_i + w_{oj})$: 计算单个隐藏层节点的输出。

外层公式: $G(\sum_{j=1}^J w_{jk}(\text{隐藏层输出}) + w_{ok})$: 计算单个输出层节点的输出。将前一步得到的隐藏层所有节点的输出作为新输入。

Multilayer Network Structure

■ A two-layer neural network implements the function:

$$f(x) = \sigma\left(\sum_{j=1}^J w_{jk} \sigma\left(\sum_{i=1}^I w_{ij}x_i + w_{oj}\right) + w_{ok}\right)$$

权重 w_{ij}, w_{jk} 决定了决策边界的方向或角度

偏置项 w_{oj} 和 w_{ok} 扮演了 $m \times c$ 中 c 的角色, 给予

Output from hidden layer

决策边界平移的自由, 否则决策边界只能穿过 $(0,0)$ 点

where: $\mathbf{x}$ is the input vector, $\mathbf{x}$ 是输入向量

w_{oj} and w_{ok} are the bias terms, 偏置项

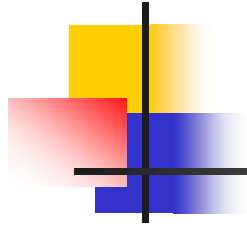
w_{ij} are the weights connecting the input with the hidden nodes

w_{jk} are the weights connecting the hidden with output nodes

σ is the sigmoid activation function.

w_{ij} 是连接输入层与隐藏层的权重, 而 w_{jk} 是连接隐藏层与输出层的权重

Multilayer Network Structure



- The hidden units enable the multilayer network to learn complex tasks by extracting *progressively more meaningful information* from the input examples.
- The MLP has a highly connected topology since every input is connected to all nodes in the first hidden layer, every unit in the hidden layers is connected to all nodes in the next layer, and so on.
- The input signals, initially these are the input examples, propagate through the neural network in a forward direction on a layer-by-layer basis, that is why they are often called *feedforward multilayer networks*. 前馈多层网络

表示能力

Representation Power of MLP

- Properties concerning the representational power of MLP: 一个足够大的两层神经网络在理论上拥有极强大的能力, 可以学习几乎所有类型的函数, 这是近似定理 (universal approximation theorem).
 - **learning arbitrary functions:** any function can be learned with an arbitrary accuracy by a two-layer network; 一个两层网络可以以任何精度学习任何函数
 - **learning continuous functions:** every bounded continuous function can be learned with a small error by a two-layer network (the number of hidden units depends on the function to be approximated); 一个两层网络可以以很小的误差学习每一个有界的连续函数
所需的隐藏单元数量取决于需要近似的函数复杂性
函数越复杂, 隐藏层越密
 - **learning Boolean functions:** every Boolean function can be learned exactly by a two-layer network although the number of hidden units grows exponentially with the input dimension. 一个两层网络可以精确地学习每一个布尔函数
但所需的隐藏单元数量会随着输入维度的增加呈指数级增长

Backpropagation Learning

Algorithm 反向传播学习算法

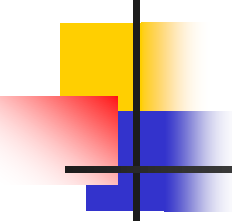
- MLP became applicable on practical tasks after the discovery of a supervised training algorithm, the **error backpropagation learning algorithm**. 该算法是MLP能实际应用的关键
- The error backpropagation algorithm includes two passes through the network:
 - **forward pass**, and 前向传播
 - **backward pass**. 反向传播

Backpropagation Learning Algorithm

反向传播的核心原理：梯度下降，它的目标是通过调整权重来最小化一个成本函数 E （通常是期望输出与网络实际输出之间的均方差）；该算法通过让权重向着误差梯度的反方向移动来实现，因为这样是误差最陡峭、最快速下降的方向。这个梯度由成本函数 E 对网络中所有权重 $w_0, w_1, \dots, w_d$ 的偏导数构成：

- During the backward pass the weights are adjusted in accordance with the **error correction rule**. The actual network output is subtracted from the given output in the example and the weights are adjusted so as to make the network output close to the desired one.
- The backpropagation algorithm does gradient descent as it moves in direction opposite to the gradient of the error, that is in direction of the steepest decrease of the error.
- This is the direction of most rapid error decrease by varying all the weights simultaneously:

$$\nabla E(\mathbf{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_d} \right] \leftarrow$$



Backpropagation Learning Algorithm

$$\nabla E(\mathbf{w}) = \left[\frac{\partial E}{\partial \mathbf{w}_0}, \frac{\partial E}{\partial \mathbf{w}_1}, \dots, \frac{\partial E}{\partial \mathbf{w}_d} \right]$$

- By gradient descent search, backpropagation training algorithm minimizes a cost function E (the mean square difference between the desired and actual net outputs).
- The network is trained initially selecting small random weights and then presenting all training data incrementally.
- Weights are adjusted after every trial using side ^{权重在每一次处理一个样本后都会被调整} information specifying the correct class until weights 整 converge and the cost function is reduced to an acceptable value.

Backpropagation Training Algorithm

初始化: 准备好训练数据集 $\{(x_e, y_e)\}$ 将网络中所有的权重 w_i 初始化为较小随机值

- **Initialization:** Examples $\{(x_e, y_e)\}_{e=1}^N$, initial weights w_i set to small random values, learning rate η 设定一个学习率
- **Repeat**
 - For each training example (x, y)

Forward 前向传播: 对于每一个训练样本, 信息首先从输入层向前传播, 计算每层神经元的输出值.

➤ *calculate the outputs* using the sigmoid function:

对于隐藏层中的每个神经元 j , 我们计算其输入的加权和 s_j , 然后将结果 s_j 通过 Sigmoid 激活函数得到神经元输出 o_j .

$$o_j = \sigma(s_j) = \frac{1}{1 + e^{-s_j}}, s_j = \sum_{i=0}^d w_{ij} o_i$$

where $o_i = x_i$ o_i 是原始输入 x_i

at the hidden units j

$$o_k = \sigma(s_k) = \frac{1}{1 + e^{-s_k}}, s_k = \sum_{j=0}^d w_{jk} o_j$$

at the output units k

对于输出层中的每个神经元 k , 我们使用上一层的输出 o_j 作为输入, 重复同样的过程, 得到网络最终输出 o_k .

Backpropagation Training

Algorithm

反向传播: 在得到网络的最终输出 o_k 后, 我们将其与真实标签 y_k 进行比较, 然后将产生的误差从后向前传输, 以计算每个权重应该更新多少。

Backward

- compute the **benefit** β_k at the node k in the output layer: 首先计算输出层每个节点 k 的误差信号

$$\beta_k = o_k (1 - o_k) [y_k - o_k]$$

effects from the output nodes

- compute the **changes for weights** $j \rightarrow k$ on connections to nodes in the output layer:

计算指向输出层的权重更新量 Δw_{jk}
连接隐藏层和输出层

$$\Delta w_{jk} = \eta \beta_k o_j$$

effects from the output of the neuron

$$\Delta w_{0k} = \eta \beta_k$$

计算隐藏层的误差信号 β_j (误差反向传播):

- compute the **benefit** β_j for the hidden node j with the formula:

$$\beta_j = o_j (1 - o_j) \left[\sum_k \beta_k w_{jk} \right]$$

隐藏层节点 j 的误差信号, 是通过它所连接的所有下一层节点 (输出层) 的误差信号 β_k 按连接权重 w_{jk} 加权求和得到的。

effects from multiple nodes in the next layer



Backpropagation Training Algorithm

- compute the **changes for the weights $i \rightarrow j$** on connections to nodes in the hidden layer:

计算指向隐藏层的权重更新量 Δw_{ij}

$$\Delta w_{ij} = \eta \beta_j o_i$$

$$\Delta w_{0j} = \eta \beta_j$$

Update

update the weights by the computed changes:

$$W = W + \Delta W$$

将变化量应用到网络的权重上

until *termination condition is satisfied.*

On-line Training

Revision by example is called *on-line (incremental) learning*.

- **Initialization:** Examples $\{(x_e, y_e)\}_{e=1}^N$, initial weights w_i set to small random values, learning rate η
- **Repeat**
 - pick a training example (x, y)
 - forward propagate the example and calculate the outputs using the sigmoid function
 - backward propagate the error to calculate the benefits
 - update the weights by the computed changes:
$$w = w + \Delta w$$
- **until** termination condition is satisfied.

数学知识不会被考到

Derivation of Backpropagation

Algorithm BP是计算误差E对网络中任意权重w的偏导数 $\frac{\partial E_e}{\partial w}$

- The BP training algorithm for MLP is a generalized gradient descent rule, according to which with each training example every weight is updated as:

$$W = W + \Delta W$$

where: $\Delta W = -\eta \frac{\partial E_e}{\partial w}$, $E_e = \frac{1}{2} \sum_k (y_k - o_k)^2$

- The implementation of the generalized gradient descent rule requires to derive an expression for the computation of the derivatives $\partial E_e / \partial w$

使用链式法则将 $\frac{\partial E_e}{\partial w}$ 分解成两部分

$$\frac{\partial E_e}{\partial w} = \frac{\partial E_e}{\partial s} \cdot \frac{\partial s}{\partial w}$$



Derivation of Backpropagation Algorithm

- The first part $\partial E_e / \partial s$ reflects the change of the error as a function of the change in the network weighted input to the unit.
- The second part $\partial s / \partial w$ reflects the change in the network weighted input as a function of the change of particular weight w to that node.
- Since:
$$\frac{\partial s}{\partial w} = \frac{\partial(\sum_l w_l o_l)}{\partial w} = o$$
- The expression is reduced as follows:

$$\frac{\partial E_e}{\partial w} = \frac{\partial E_e}{\partial s} \cdot o$$

Derivation of Backpropagation Algorithm

- For weights $j \rightarrow k$ on connections to nodes in the output layer:

输出层!

$$\frac{\partial E_e}{\partial w_{jk}} = \frac{\partial E_e}{\partial s_k} \cdot o_j$$

$$\frac{\partial E_e}{\partial s_k} = \frac{\partial E_e}{\partial o_k} \cdot \frac{\partial o_k}{\partial s_k}$$

$$\frac{\partial E_e}{\partial o_k} = \frac{\partial(\frac{1}{2} \sum_k (y_l - o_l)^2)}{\partial o_k} = \frac{\partial(\frac{1}{2} (y_k - o_k)^2)}{\partial o_k}$$

$$= \frac{1}{2} \cdot 2 \cdot (y_k - o_k) \frac{\partial (y_k - o_k)}{\partial o_k}$$

$$= -(y_k - o_k)$$

$$\frac{\partial o_k}{\partial s_k} = \frac{\partial \sigma(s_k)}{\partial s_k} = o_k (1 - o_k)$$

Derivation of Backpropagation Algorithm

- Therefore:

$$\frac{\partial E_e}{\partial s_k} = -(y_k - o_k) o_k (1 - o_k)$$

$$\frac{\partial E_e}{\partial w_{jk}} = \frac{\partial E_e}{\partial s_k} \cdot o_j$$

- Then we substitute:

$$\Delta w_{jk} = -\frac{\partial E_e}{\partial w_{jk}} = \eta \beta_k o_j$$

$$\beta_k = \frac{\partial E_e}{\partial s_k} = -(y_k - o_k) o_k (1 - o_k)$$

- The gradient descent rule in previous lecture:

$$\Delta w_i = \Delta w_i + \eta (y_e - o_e) \sigma(s) (1 - \sigma(s)) x_{ie}$$

Derivation of Backpropagation Algorithm

- For weights $i \rightarrow j$ on connections to nodes in the hidden layer

$$\frac{\partial E_e}{\partial w_{ij}} = \frac{\partial E_e}{\partial s_j} \cdot o_i$$

隐藏层！

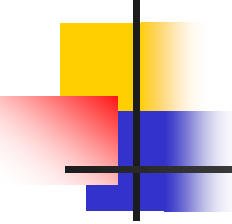
- In this case the error depends on the errors committed by all output units:

$$\begin{aligned} \frac{\partial E_e}{\partial s_j} &= \sum_k \frac{\partial E_e}{\partial s_k} \cdot \frac{\partial s_k}{\partial s_j} = \sum_k -\beta_k \cdot \frac{\partial s_k}{\partial s_j} \\ &= \sum_k -\beta_k \cdot \frac{\partial s_k}{\partial o_j} \cdot \frac{\partial o_j}{\partial s_j} \\ &= \sum_k (-\beta_k) \cdot w_{jk} \cdot \frac{\partial o_j}{\partial s_j} = \sum_k (-\beta_k) \cdot w_{jk} \cdot o_j (1 - o_j) \end{aligned}$$

$$\frac{\partial E_e}{\partial s_k} = -(y_k - o_k) o_k (1 - o_k)$$

$$o_k = \sigma(s_k) = \frac{1}{1 + e^{-s_k}}, s_k = \sum_{i=0}^d w_{jk} o_i$$

$$o_j = \sigma(s_j) = \frac{1}{1 + e^{-s_j}}, s_j = \sum_{i=0}^d w_{ij} o_i$$



Derivation of Backpropagation Algorithm

- For the hidden units:

$$\left\{ \begin{array}{l} \Delta w_{ij} = \eta \beta_j o_i \\ \Delta w_{0j} = \eta \beta_j \end{array} \right. \quad \underline{\beta_j = -\frac{\partial E_e}{\partial s_j} = o_j (1 - o_j) [\sum_k \beta_k w_{jk}]}$$

Note: This analysis was made for a single training pattern, but it can be generalized so that:

$$\frac{\partial E_{total}}{\partial w_{ij}} = \sum_e \frac{\partial E_e}{\partial w_{ij}}$$

Thus, we just need to sum out weight changes over the examples.

Batch Backpropagation Algorithms

批量反向传播核心思想
是按轮次修正 (revision by epoch)

Revision by Epoch

按数学角度来说, 最准确的梯度计算应该在处理完一个 epoch (轮次) 之后, 也就是在处理完训练集中所有样本之后才进行, 最后用于更新权重的误差导数, 是所有单个样本误差导数总和

- From mathematical point of view the error derivatives should be computed after each epoch, *i.e.*, after all examples in the training set have been processed. 但是在实践中, 之前提到的单个样本修正的增量学习效果更好, 更常用
- This means that the error derivative is taken to be the sum of the error derivatives for all examples.
- While this revision by epoch may have stronger theoretical motivation, revision by a particular example may yield better results and is more commonly used.
- Revision by epoch is called **Batch Learning**.

Batch version of the backpropagation algorithm

- **Initialization:** 准备好训练样本集 $\{(x_e, y_e)\}$, 将权重 w_i 初始化为较小随机值
Examples $\{(x_e, y_e)\}_{e=1}^N$, initial weights w_i set to small random values, learning rate η
- **Repeat** 循环核心是先计算, 后更新
 - for each training example (x, y) 遍历每一个训练样本 (x, y)
 - forward propagate the example and calculate the outputs using the sigmoid function 对每个样本执行前向传播, 计算网络输出
 - backward propagate the error to calculate the benefits 对每个样本执行反向传播, 计算误差信号 (benefits) 以及对应的权重变化 Δw
 - after processing all examples update the weights by the computed changes: 在这一步计算出 Δw 会被累加, 而不是立即应用
在处理完所有样本之后, 用累加得到的总变量 Δw 来对权重进行一次更新
- **until** termination condition is satisfied. 遍历所有样本 $\rightarrow$ 更新一次权重

Issues of Backpropagation

Presentation order of training samples

训练样本的呈现顺序：在使用增量/在线学习时，应该以什么样的顺序将训练数据喂给神经网络。

一轮轮次(epoch)代表我们的算法完整地处理了一遍训练集中的所有样本。训练的长度通常是用完成了多少个轮次来衡量的。

Sequential or random presentation

顺序呈现：每轮训练样本都按相同、固定顺序呈现

- The epoch is the fundamental unit for training, and the length of training often is measured in terms of epochs.
- During a training epoch with revision after a particular example, the examples can be presented in the same sequential order, or the examples could be presented in a different random order for each epoch.
- The random presentation usually yields better results

顺序呈现往往能呈现更好的结果

顺序呈现往往包含一些“伪规律”，因为训练样本每次都按固定规律传入

Issues of Backpropagation:

Initialization

训练开始时, 网络中所有权重都会被初始化为一些随机数, 一个典型的范围是在 -0.5 到 0.5 之间。
随机的初始状态。

Random initial state 通常输入数据也会被归一化到 0 和 1 之间

- Unlike many other learning systems, the neural network begin in a random state. The network weights are initialized to some choice of random numbers with a range typically between -0.5 and 0.5 (the inputs are usually normalized to numbers between 0 and 1).
- Even with identical learning conditions, the random initial weights can lead to results that differ from one training session to another. 即使在完全相同学习条件下, (相同网络结构, 学习率和训练数据), 随机的初始权重也可能导致每一次训练得到的结果有所不同。

Issues of Backpropagation:

Hidden layer 应该设置多少个隐藏层

增加额外的隐藏层并不会增加网络在判别问题上的表示能力; 虽然两层隐藏层的网络更

- While it may be more convenient to specify more than one layer of hidden units, *additional layers do not add representational power* to the discrimination. 强大, 但对于实践中来说, 单层隐藏层网络可能已经足够精准

- Two-hidden-layer networks are more powerful, but one-hidden-layer networks may be sufficiently accurate for many tasks 并且单层隐藏层训练速度更快

- One-hidden layer networks assume faster training.

如果使用单层隐藏层, 需要考虑隐藏层中应该包含多少神经元

- A heuristic to start with:

One hidden layer, with n hidden neurons,
 $n = (\text{inputs} + \text{output_neurons}) / 2$

神经元数量 $n = (\text{输入层神经元数量} + \text{输出层神经元数量}) / 2$

Issues of Backpropagation:

Stopping Criteria 停止训练的条件

- The stopping criteria is checked at the end of each epoch:
 - ① 误差低于预定的阈值, 则停止训练, 通常是根据经验决定的
 - The error (mean absolute or mean square) at the end of an epoch is below a threshold 这种方法是在一个轮次之后, 计算模型在整个训练集上的平均误差
 - All training examples are propagated and the mean (absolute or square) error is calculated
 - The threshold is determined heuristically – e.g. 0.01
 - Maximum number of epochs is reached ② 达到最大轮次: 设定一个最大的训练轮次数
 - Early stopping using a validation set (explain later) ③ 使用验证集进行早停: 这是一种更高级的策略, 它会使用一个独立的验证集来决定何时停止
- It typically takes hundreds or thousands of epochs for an NN to converge 如果一个神经网络需要收敛, 通常要数百甚至数千个轮次的训练。

Issues of Backpropagation: Learning rate

尽管反向传播在训练数据上可以获得很好的拟合效果,但在将其应用于独立测试数据的性能预测时,会充满困难和陷阱。正确设置学习率是克服这些困难的第一步。

- While it is possible to get excellent fits to training data, the application of backpropagation is fraught with difficulties and pitfalls for the prediction of the performance on independent test data.
- Many choices to be made in applying the gradient descent method. 学习率的选择对于能否找到误差曲线上的真正全局最小值
- The key variations of these choices are : 至关重要.

the learning rate and local minima

the selection of a learning rate is of critical importance in finding the true global minimum of the error distance.

The learning rate and local minima

学习率太小: 训练过程的进展极其缓慢,

学习率太大: 训练初期进展会很快, 但可能会在两个相对较差的解之间

来回振荡, 无法收敛到最优解



例如步太大, 导致在两个山峰来回跳转

- Backpropagation training with too small a learning rate will make agonizingly slow progress. Too large a learning rate will proceed much faster, but may simply produce oscillations between relatively poor solutions.
- Both of these conditions are generally detectable through experimentation and sampling of results after a fixed number of training epochs. 使用那个在保证能收敛到最小解的前提下, 尽可能大的学习率.
- Typical values for the learning rate parameter are numbers between 0 and 1:

$$0.05 < \eta < 0.75$$

- One would like to use the largest learning rate that still converges to the minimum solution.

Issues of Backpropagation:

Momentum 动量

动量主要是稳定权重的变化, 普通的梯度下降就像一个没有质量的小球完全根据当前

地面的坡度来决定下一步的方向, 这容易导致震荡

- The momentum is to stabilize the weight change using a combination of the gradient decreasing term with a fraction of the previous weight change:

$$\Delta w(t) = -\eta \frac{\partial E_e}{\partial w(t)} + \alpha \Delta w(t-1)$$

动量法则通过赋予系统一定的惯性 (inertia), 权重的更新方向会倾向于和上一步相同的方向, 除非当前梯度强烈反对这个方向

- where t is the index of the current weight change, 反对这个方向
- This gives the system a certain amount of inertia since the weight vector will tend to continue moving in the same direction unless opposed by the gradient term.

$\propto$ 控制惯性大小, 直接影响了训练动态

Issues of Backpropagation:

Momentum

为了实现惯性, 动量法修改了权重的更新规则,

- ◆ Momentum term simply makes the following change to the weight update rule, where α is the momentum term:
新的更新量 $\Delta w(t)$ 由两部分组成: 一部分是当前的梯度下降项, 另一部分是前一次权重更新量 $\Delta w(t-1)$ 的一部分
- If $\alpha=0$, this is the same as the regular backpropagation, where the weight update is determined purely by the gradient descent $\alpha=0$: 动量项为0, 更新规则退化为常规的反向传播算法, 更新由当前梯度决定
- If $\alpha=1$, the gradient descent is completely ignored, and the update is based on the 'momentum', previous weight update rule 如果 $\alpha=1$, 此时梯度下降被完全忽略, 更新主要基于前一次权重更新
- Typical value for α is generally between 0.6 and 0.9 α 通常 $0.6 \leq \alpha \leq 0.9$

$$\Delta w(t) = -\underbrace{\eta \frac{\partial E_e}{\partial w(t)}}_{\text{梯度下降项}} + \underbrace{\alpha \Delta w(t-1)}_{\text{动量项}} \quad (1-\alpha)$$

Issues of Backpropagation: Momentum

平滑更新并抑制振荡

平滑权重变化 抑制交叉缝合效应, 即消除在误差山谷中来回摆动的振荡

The momentum has the following effects:

- it smooths the weight changes and suppresses cross-stitching, that is **cancels side-to-side oscillations across the error valley**; 在方向一致时加速收敛: 当所有权重更新方向都一致, 动量会放大等效
- when all weight changes are all in the same direction the **learning rate** momentum amplifies the learning rate causing a faster **convergence**; 使更快收敛
- enables to escape from small local minima on the error surface. 帮助逃离局部最小值: 动量能够帮助算法从误差曲面上的小型局部最小值中逃逸出来

Issues of Backpropagation:

泛化和过拟合

Generalization & Overfitting

监督学习: 我们使用一个包含正确行为的有限数量的样本 (p_i, t_i) 来进行训练

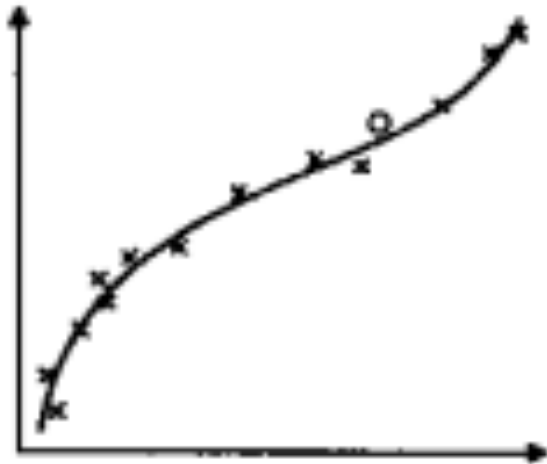
- Supervised learning – training with finite number of examples of proper behavior: $\{ p_i, t_i \}, i=1, \dots, n$
- Based on them the network should be able to *generalize* what it has learned to the total population of examples
基于这些有限的训练样本, 网络的最终目标是能够将其所学到的知识泛化到所有可能的样本总体中去。
- **Overtraining (overfitting):**
泛化失败
表现力: 在训练集上表现优异但在新数据上, 误差很高
- the error on the training set is very small but when a new data is presented to the network, the error is high
- → the network has memorized the training examples but has not learned to generalize to new situations!

When Does Overfitting Occur?

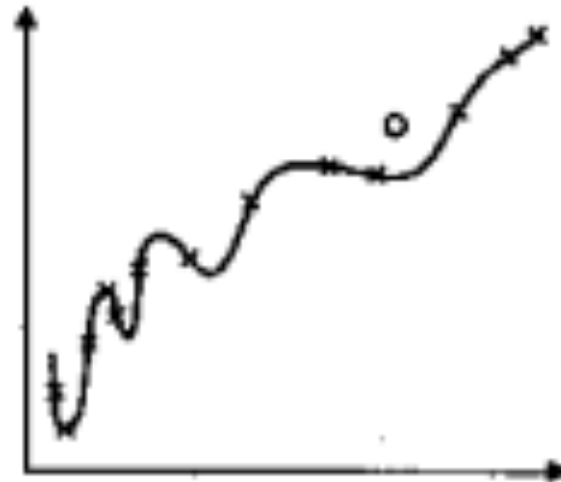
- Training examples are noisy

Example: x- training set, o-testing set

A good fit to noisy data



Overfitting



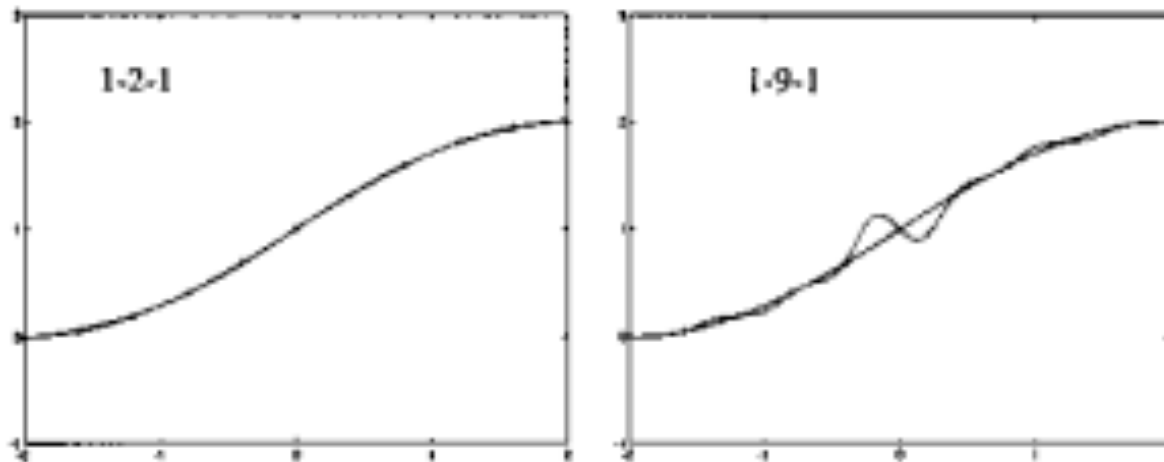
When Does Overfitting Occur? – cont.

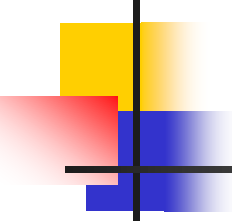
- **Common reason:** number of the free parameters is bigger than the number of training examples

自由参数数量大于训练样本数量 (一个强大的模型学习

小的数据集)

$f(x) = 1 + \sin\left(\frac{6\pi}{4}x\right)$ was sampled to create 11 training examples





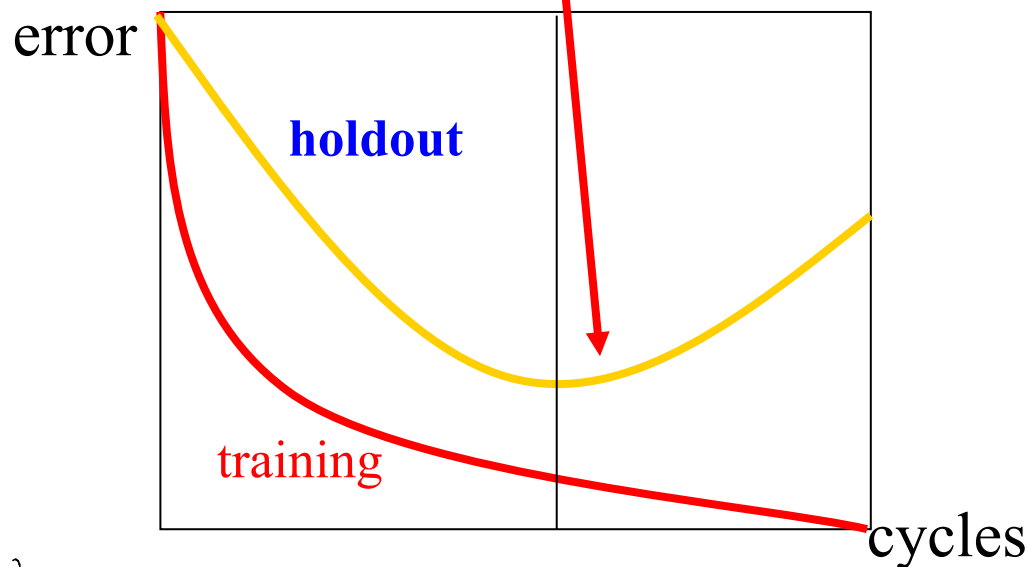
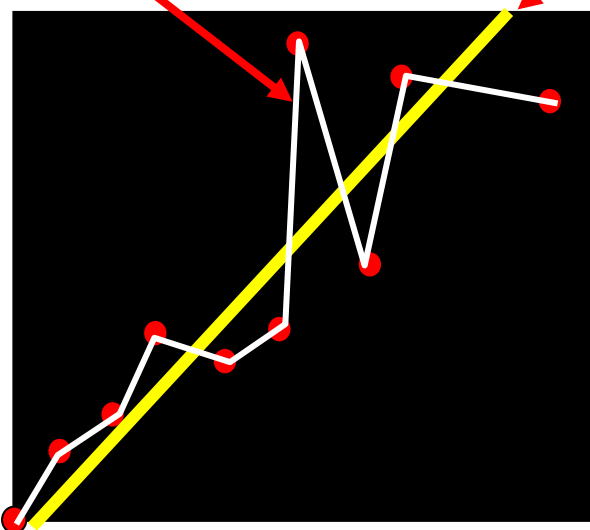
Preventing Overtraining

① 使用一个大小恰好的网络：自由参数的数量不能超过训练样本数量
但在实践中很难预知一个特定任务需要多大的网络

- Use network that is just large enough to provide an adequate fit
 - Don't use a bigger network when a smaller one will work
 - The network should not have more free parameters than there are training examples
- However, it is difficult to know beforehand how large a network should be for a specific application

Issues of Backpropagation: Generalization & Overfitting

Overfitted model “Real” model Overfitted model



0

其他技术包括：网络剪枝，应用正则化技术

- With sufficient nodes MLP can classify any training set exactly;
- But it may have poor generalisation ability.
- **Overfitting** may be prevented by **early stopping**, **network pruning**, and applying **regularization techniques**.

Techniques to overcome overfitting

权重衰减 (正则化技术): 每次迭代过程中, 将每个权重都减小一个小的因子.

- **Weight decay**: **Decrease** each weight by some small factor during each iteration.
- The **motivation**: to keep weight values small. 保持权重值较小,
- Add penalty term to the error function 通过向误差函数中添加一个
- ✓ Penalizes large weights to reduce variance 惩罚项来实现, 这个惩罚项
- ✓ Standard weight decay equation 会对较大权重惩罚, 降低模型方差

最终, 权重衰减的惩罚项会使得权重收敛到比正常情况下更小的绝对值。

Techniques to overcome overfitting

阻止学习过程 (主要风险): 大权重会导致神经元的加权输入 s 值很大, 这会把非线性激活函数 (如Sigmoid) 的输出推向饱和的平坦区域 (输出接近0或1的地方)。在这些平坦区域, 函数的导数几乎为零。在反向传播中, 这会导致梯度消失, 从而使得这些神经元的學習过程被不可逆地停止。

导致输出异常: 如果输出激活函数没有边界, 指向输出单元的权重过大可能会导致输出远远超出正常数据范围的“疯狂”结果。

- Weight decay penalty term causes the weights to converge to smaller absolute values than they otherwise would.
权重衰减惩罚项会使权重收敛至比未施加惩罚时更小的绝对值。
- Large weights can hurt generalization in two different ways.
 - 导致函数过于粗糙: 指向隐藏单元的权重过大, 可能会导致输出函数过于“崎岖”, 甚至出现接近不连续的剧烈变化。
 - Excessively large weights leading to hidden units can cause the output function to be too rough, possibly with near discontinuities.
 - Excessively large weights leading to output units can cause wild outputs far beyond the range of the data if the output activation function is not bounded to the same range as the data.
- The main risk with large weights is that the non-linear node outputs could be in one of the flat parts of the transfer function, where the derivative is zero. In such case the learning is irreversibly stopped.

交叉验证是指在训练数据之外，额外使用一组验证数据 (validation data)。

Techniques to overcome overfitting

早停法的具体流程:

训练的目标是找到能够使验证集误差最低的迭代次数。

为此，算法会保留两份权重：一份是当前正在训练的权重，另一份是迄今为止在验证集上表现最好的权重副本。

一旦当前训练的权重在验证集上的误差，比已经储存的最佳权重的误差还要高，训练就会被终止，并返回储存的最佳权重作为最终结果。

- **Cross-validation:** a set of **validation data** in addition to the training data.
- The algorithm **monitors** the error *w.r.t.* this validation data while using the training set to drive the gradient descent search. 算法使用训练集来驱动梯度下降的搜索过程（即计算梯度和更新权重）。同时，算法会持续监控模型在验证集上的误差。
 - How many weight-tuning iterations should the algorithm perform? It should use the number of iterations that produces the lowest error over the validation set.
 - Two copies of the weights are kept: one copy for training and a separate copy of the best weights thus far, **measured by their error over the validation set**.
 - Once the trained weights reach a higher error over the validation set than the stored weights, training is terminated and the stored weights are returned.

使用单个固定验证集方法的局限性，尤其是在数据集很小的情况下。数据不足: 可能没有足够的数据来分割出一个独立的验证集，因为这样做会显著减少用于训练的数据量，从而影响模型学习； 过拟合风险: 而过拟合问题在小数据集上恰恰最为严重。这就产生了一个两难的局面：我们最需要验证来防止过拟合，但又最承受不起分割数据所带来的损失。

K-fold cross validation

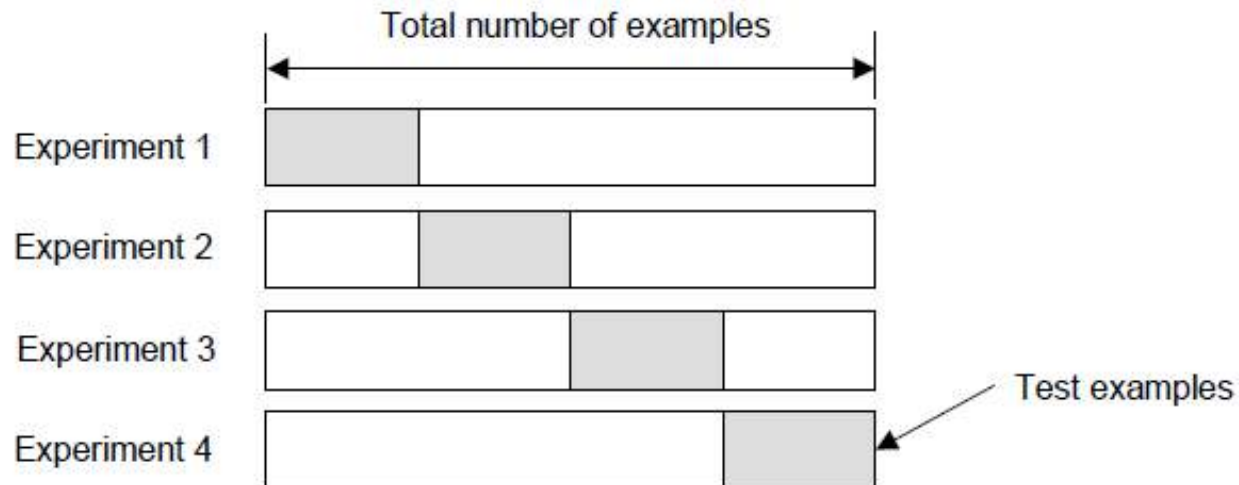
原始的样本集被随机地分割成 K 个子样本集（也称为“折”）。

整个交叉验证过程会重复 K 次。在每一次实验中：一个子样本集被保留作为验证数据（用于测试模型）。剩下的 $K-1$ 个子样本集被用作训练数据。关键在于，每一个子样本集都会被精确地用作一次验证数据

- Problems with the validation set approach – small data sets^① 为每一折找到最佳轮次: 在 K 次实验的每一次中，我们都独立地训练模型，并找出能使模型在该折的测试（验证）部分表现最佳的训练轮次数 ep 。
 - Not enough data may be available to provide a validation set^② 计算平均值: 在完成所有 K 次实验后，我们会得到 K 个最佳轮次数。然后我们计算这些值的平均值 ep_{mean} 。
 - Overfitting is most severe for small data sets!
- K-fold cross validation may be used
 - Each time determine the number of epochs ep that result in best performance on the respective test partition
 - ③ 最终训练: 最后，我们使用全部的训练数据（不再分割），将网络从头开始训练 ep_{mean} 个轮次。这样得到的最终模型，就是我们用来进行实际预测的模型。
 - Calculate the mean of ep : ep_{mean}
 - Final run: train the network on all examples for ep_{mean} epochs

More on k-fold cross-validation

- In K -fold cross-validation, the original sample is randomly partitioned into K subsamples. Of the K subsamples, a single subsample is retained as the validation data for testing the model, and the remaining $K - 1$ subsamples are used as training data.
- **The cross-validation process is then repeated K times (the *folds*), with each of the K subsamples used exactly once as the validation data.**

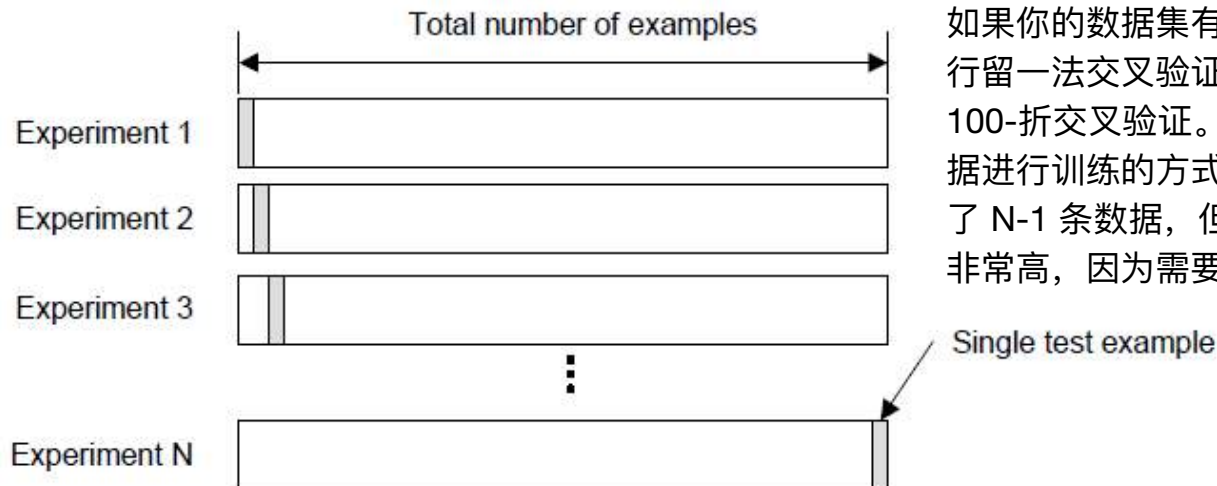


留一法交叉验证 (LOOCV) 是指，从原始样本中只使用单个观测（一条数据）作为验证数据，而所有其余的观测都作为训练数据。这个过程会不断重复，直到样本中的每一个观测都轮流作过一次验证数据为止。

Leave-one-out cross-validation

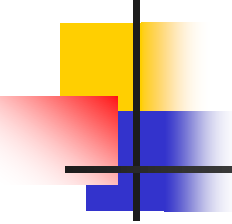
假设总共有 N 个样本，那么就要进行 N 次实验。第一次实验用第1个样本做测试，第二次用第2个，以此类推，直到第 N 次实验用第 N 个样本做测试。

- leave-one-out cross-validation (LOOCV) involves **using a single observation from the original sample as the validation data, and the remaining observations as the training data.** 留一法交叉验证，实际上等同于将 K -折交叉验证中的 K 值，设定为原始样本中观测的总数量。
- This is repeated such that each observation in the sample is used once as the validation data.
- This is the same as a K -fold cross-validation with K being equal to the number of observations in the original sample.



如果你的数据集有100条数据，那么对它进行留一法交叉验证，就等同于进行了一次100-折交叉验证。这是一种最大化利用数据进行训练的方式，因为每次训练都使用了 $N-1$ 条数据，但它的计算成本也相应地非常高，因为需要训练 N 个模型。

Limitations & Capabilities of MLP



理论上, MLP可以执行任何线性和非线性的映射。

它们能够以任意精度逼近任何合理的函数。

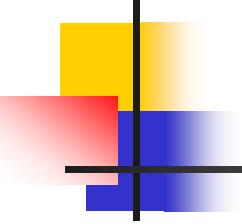
正是这种强大的表示能力, 使得MLP能够克服单层感知机的局限性。

- MLPs trained with backpropagation can perform function approximation and pattern classification
- Theoretically they can
 - Perform any linear and non-linear mapping
 - Can approximate any reasonable function arbitrary well
 - => Overcome the limitations of perceptrons
- In practice:
 - May not always find a solution – can be trapped in a **local minima**
 - The performance is sensitive to the starting conditions (initialization of weights)

局部最小值 (Local Minima): MLP在训练时不一定总能找到一个解决方案, 因为它可能会被困在误差曲面的局部最小值中。

对初始化的敏感性: 网络的最终性能对起始条件 (即权重的随机初始化) 非常敏感。从不同的随机点开始训练, 可能会得到性能不同的模型。

Limitations & Capabilities of MLP



对隐藏层和神经元数量的敏感性：

神经元太少: 会导致欠拟合 (underfitting), 即模型过于简单, 无法学习到想让它学习的规律。

神经元太多: 会导致过拟合 (overfitting), 并且训练速度缓慢。

隐藏层和神经元的数量选择, 最终都取决于设计者。

■ In practice:

- Sensitive to the number of hidden layers and neurons
 - Too few neurons – underfitting, unable to learn what you want it to learn
 - too many – overfitting, learns slowly
 - the number of hidden layers and neurons are left to the designer
- Sensitive to the value of the learning rate
 - Too small – slow learning
 - Too big – instability or poor performance
 - The proper choices depends on the nature of examples

对学习率的敏感性：

学习率太小: 会导致学习速度缓慢。

学习率太大: 会导致训练过程不稳定或性能不佳。

正确的选择取决于具体问题的特性。

■ Trial and error

- Refer to the choices that have worked well in similar problems
- Successful applications of NNs requires time and experience



THANK YOU



VISIT US

WWW.XJTLU.EDU.CN



FOLLOW US

@XJTLU



Xi'an Jiaotong-Liverpool University

西交利物浦大學